

MISSÃO AURORA SIGER

REGRAS LÓGICAS DO NCAS

Expressões booleanas, simplificação e Teoremas de De Morgan

Documento técnico da regra de decisão utilizada pelo Núcleo Cognitivo da Aurora Siger.

EQUIPE

Juan de Lucas Frois - RM563260
Flávia Roberta Pennachin - RM561860
Pedro Valente Toledo - RM570394
Renan Mano Otero - RM573615

FIAP - Ciência da Computação - Turma 1CCOA - 2026

1. Finalidade da regra

O NCAS precisa identificar quando uma ocorrência deve gerar alerta operacional. A regra considera uma falha confirmada e, de forma complementar, o consumo elevado de um módulo essencial. A simplificação reduz a expressão sem alterar qualquer resultado.

2. Variáveis booleanas

Símbolo	Variável	Valor verdadeiro quando
F	Falha	Existe uma falha confirmada no módulo.
C	Crítico	A ocorrência foi classificada como crítica.
E	Consumo elevado	O consumo está acima do limite operacional.
M	Módulo essencial	O módulo é indispensável à segurança ou à vida.

3. Expressão original

$A = (F \text{ AND } C) \text{ OR } (F \text{ AND NOT } C) \text{ OR } (E \text{ AND } M)$

Os dois primeiros termos cobrem uma falha crítica e uma falha não crítica. O terceiro termo cobre consumo elevado em módulo essencial.

4. Simplificação passo a passo

Etapa	Transformação	Teorema ou identidade
1	$A = (F \text{ AND } C) \text{ OR } (F \text{ AND NOT } C) \text{ OR } (E \text{ AND } M)$	Expressão original
2	$A = F \text{ AND } (C \text{ OR NOT } C) \text{ OR } (E \text{ AND } M)$	Distributividade
3	$A = F \text{ AND } 1 \text{ OR } (E \text{ AND } M)$	Complemento: $C \text{ OR NOT } C = 1$
4	$A = F \text{ OR } (E \text{ AND } M)$	Identidade: $F \text{ AND } 1 = F$

A forma simplificada exige menos operações e mantém exatamente a mesma tabela-verdade.

5. Verificação por tabela-verdade

As dezesseis combinações possíveis de F, C, E e M foram avaliadas. A coluna de equivalência permanece verdadeira em todas as linhas.

F	C	E	M	Original	Simplificada	Equivale?
0	0	0	0	0	0	SIM
0	0	0	1	0	0	SIM
0	0	1	0	0	0	SIM
0	0	1	1	1	1	SIM
0	1	0	0	0	0	SIM
0	1	0	1	0	0	SIM
0	1	1	0	0	0	SIM
0	1	1	1	1	1	SIM
1	0	0	0	1	1	SIM
1	0	0	1	1	1	SIM
1	0	1	0	1	1	SIM
1	0	1	1	1	1	SIM
1	1	0	0	1	1	SIM
1	1	0	1	1	1	SIM
1	1	1	0	1	1	SIM
1	1	1	1	1	1	SIM

Resultado: A_original = A_simplificada para 16 de 16 combinações. A variável C continua útil para definir a prioridade, mas deixa de ser necessária para decidir se uma falha gera alerta.

6. Regra de bloqueio com De Morgan

O sistema também deve bloquear uma operação quando não for possível confirmar simultaneamente a segurança e a consistência dos dados.

B = NOT (S AND D)
B = (NOT S) OR (NOT D)

S	D	NOT(S AND D)	(NOT S) OR (NOT D)	Ação
0	0	1	1	Bloquear
0	1	1	1	Bloquear
1	0	1	1	Bloquear
1	1	0	0	Permitir análise

Se qualquer condição falhar, o NCAS interrompe a automação e solicita revisão humana.

7. Implementação em Python

A implementação utiliza os operadores lógicos nativos do Python. A forma original é preservada para demonstração e comparada à forma simplificada durante os testes.

```
def regra_original(f, c, e, m):
    return (f and c) or (f and not c) or (e and m)

def regra_simplificada(f, e, m):
    return f or (e and m)

def regra_bloqueio(seguranca_ok, dados_consistentes):
    return (not seguranca_ok) or (not dados_consistentes)
```

8. Casos operacionais

Caso	Valores relevantes	Resultado	Prioridade
Falha crítica em oxigênio	F=1, C=1	Alerta	Crítica
Falha não crítica	F=1, C=0	Alerta	Alta
Consumo alto em energia	F=0, E=1, M=1	Alerta	Alta
Consumo alto em laboratório	F=0, E=1, M=0	Sem alerta	Normal
Dados inconsistentes	S=1, D=0	Bloqueio	Crítica

9. Conclusão

A simplificação reduz a regra principal de três termos para dois, conserva o comportamento em todas as entradas e facilita auditoria. A regra de De Morgan torna explícito que segurança ou consistência insuficientes impedem uma decisão automatizada. Em ambos os casos, a lógica apoia especialistas humanos; ela não elimina sua responsabilidade.